5202 ceD 92]GL .sc[1v01332.21522:viXra

Splitwise：通过李雅普诺夫辅助深度强化学习实现大语言模型的协作边缘-云推理

阿博尔法兹尔·尤内西
Abolfazl.Younesi@uibk.ac.at 因斯布鲁克大学 奥地利 因斯布鲁克

阿巴斯·沙布朗·马里安 abbas.shabrang81@sharif.edu 沙里夫理工大学 伊朗德黑兰

伊利亚斯·奥斯塔德
elyas.oustad23@sharif.edu 伊朗德黑兰沙里夫理工大学

扎赫拉·纳贾法巴迪·萨马尼
Zahra.Najafabadi- Samani@uibk.ac.at 因斯布鲁克大学 奥地利 因斯布鲁克

穆罕森·安萨里
ansari@sharif.edu 伊朗德黑兰 沙里夫理工大学

托马斯·法林格
Thomas.Fahringer@uibk.ac.at 因斯布鲁克大学 奥地利 因斯布鲁克

摘要

在边缘设备上部署大型语言模型（LLM）具有挑战性，因为这些设备的内存和电源资源有限。仅依赖云端推理可以减轻设备负担，但会带来高延迟和高成本。静态的边缘-云分区仅优化单一指标，并且在带宽波动时表现不佳。我们提出了 Splitwise，一种新颖的结合李雅普诺夫（Lyapunov）辅助深度强化学习（DRL）的框架，用于在边缘和云环境中对 LLM 进行细粒度的自适应分区。Splitwise 将 Transformer 层分解为注意力头和前馈子模块，暴露出比按层分区方案指数级更多的分区选择。一个受李雅普诺夫优化指导的分层 DRL 策略，能够在保证队列在随机工作负载和可变网络带宽下稳定的同时，共同最小化延迟、能耗和精度损失。Splitwise 还通过分区检查点以及在通信失败情况下的指数退避恢复机制保证系统的鲁棒性。

CCS 概念：
- 计算方法 → 强化学习；
- 计算机系统组织 → 云计算；
- 信息系统 → 语言模型。

关键词：大语言模型，边缘-云推理，李雅普诺夫优化，深度强化学习

ACM 参考格式：
Abolfazl Younesi, Abbas Shabrang Maryan, Elyas Oustad, Zahra Najafabadi Samani, Mohsen Ansari 和 Thomas Fahringer. 2025. Splitwise：通过李雅普诺夫辅助深度强化学习进行大型语言模型的协作边缘-云推理。收录于 2025 年 IEEE/ACM 第 18 届国际实用与云计算会议（UCC ’25），2025 年 12 月 1–4 日，法国，法国。ACM，美国纽约，NY，共 11 页。https://doi.org/10.1145/3773274.3774267

1 引言

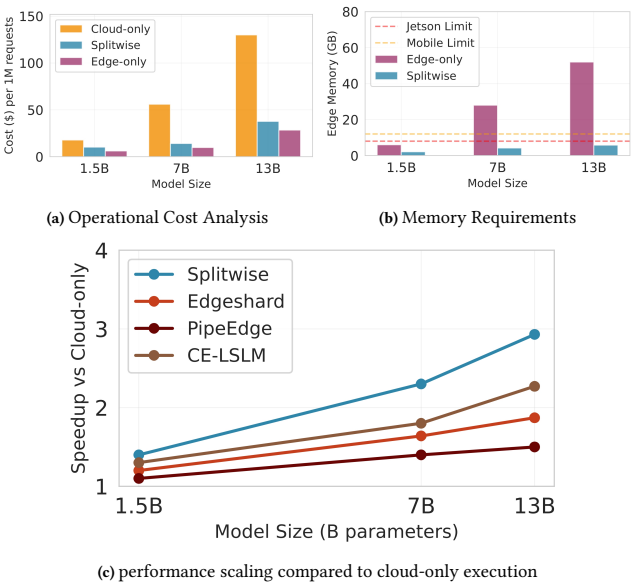
大型语言模型（LLMs）的部署已经革新了众多应用，从智能助手到代码生成 [2, 12, 32]。然而，它们的计算需求常常超过数十亿参数，这给资源受限的边缘设备带来了重大挑战。虽然基于云的推理提供了丰富的计算资源，但它会引入显著的延迟（50-200毫秒）、成本，并且对敏感应用提出隐私方面的担忧 [7, 18, 29]。这种延迟对于下一代交互式边缘应用来说是不可接受的。例如，实时对话代理和增强现实（AR）助手都需要“感知为即时”的响应（理想情况下低于100毫秒）以保持流畅的用户体验。50-200毫秒的网络往返时间在推理计算开始之前就使这一目标无法实现。此外，许多用例涉及处理敏感数据，如私人消息和医疗转录，在这种情况下，卸载数据

这一差距突显了一种协作式边缘-云推理范式，该范式策略性地利用了边缘接近性和云容量 [18, 21, 24, 34]。这些场景经常

Please use nonacm option or ACM Engage class to enable CC licenses
This work is licensed under a Creative Commons Attribution 4.0 International License.
UCC ’25, December 1–4, 2025, France, France
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2285-1/2025/12
https://doi.org/10.1145/3773274.3774267

在无线带宽（10–100 Mbps）和由于交互式用户行为引起的突发请求到达方面表现出快速变化。在这种动态情况下，静态分区很快变得不再最优甚至不稳定，导致队列积压和尾部延迟违规。因此，在基于大语言模型的服务的实际部署中，动态分区对于保持响应性和资源效率至关重要。

现有工作的局限性。当前的边缘-云协同推理方法存在三个根本性局限。首先，静态分割策略 [4, 14, 16] 预先确定了边缘与云之间的分割点，无法适应动态的网络状况和不同的工作负载特性。例如，当网络带宽在 10 到 100 Mbps 之间波动时，这在移动场景中很常见，静态方法要么无法充分利用可用带宽，要么会遭遇严重的瓶颈 [17, 32]。其次，现有方法通常以粗粒度进行模型分割（例如整个 Transformer 层），错过了细粒度优化的机会 [9, 18]。这种粗粒度分割限制了在子层级优化资源分配的能力，从而降低了在异构边缘环境中的效率 [31]。第三，以往的工作通常仅优化单一目标，如延迟或能耗，忽视了延迟与其他因素之间的复杂相互作用。



(a) Operational Cost Analysis (b) Memory Requirements

(c) performance scaling compared to cloud-only execution

图 1. 关键实验指标的示意图，突出显示 (a) 运营成本、(b) 内存需求和 (c) 与仅云端执行相比的性能扩展的变化。（设置的详细信息见第 5 节）

提出的方法。我们介绍了 Splitwise，一种新颖的框架，利用 Lyapunov 辅助深度强化学习（DRL）在边缘设备和云服务器之间动态分配大语言模型（LLM）。与现有方法不同，Splitwise 将分配问题表述为受约束的马尔可夫决策过程（MDP），其中动作决定注意力头和前馈网络子层级的细粒度分配点。Lyapunov 优化框架在队列稳定性方面提供理论保证，即使在随机到达和时变网络条件下也能确保延迟有界。我们的 DRL 智能体通过将 Lyapunov 漂移纳入奖励函数，学习在即时性能指标（延迟、能耗）与长期系统稳定性之间实现平衡。

我们展示了一些关键的权衡，这些权衡基于图1中的观察结果，定义了我们基于Splitwise的边缘-云推理格局。图1a突出了各种推理策略的运营成本。随着模型规模从15亿（B）参数增长到130亿参数，云端独立执行每百万请求的成本急剧上升，对于最大模型超过150美元。这是由于需要强大的云基础设施造成的。相比之下，仅边缘执行仍然具有成本效益，但对于较大模型受到内存限制。Splitwise显著降低了成本，即使对于130亿参数模型，通过利用边缘计算，将成本保持在25美元以下（比纯云方案低6倍）。

以及云资源。图1b显示了边缘设备的内存占用情况。13B参数模型需要超过50 GB，超出了典型移动设备8 GB的限制和大多数边缘网关16 GB的限制，使大规模推理变得困难。Splitwise通过有效的计算卸载将所需的边缘内存减少到10 GB，从而解决了这一问题。最后，图1c展示了协作推理带来的性能提升，对于13B参数模型最高可实现3倍的加速。总体而言，这些结果强调了Splitwise的优势。它们表明传统方法通常成本过高且效率低下。

见解。根据我们的观察，我们有两个关键见解。首先，大型语言模型（LLM）中不同组件的计算成本和数据传输需求差异显著。LLM中的注意力机制需要二次计算，但生成紧凑的表示，而前馈网络（FFN）则表现出相反的特性 [18, 23]。这种异质性表明，最佳分区点会根据运行时条件而不同。其次，边缘-云系统本质上表现出队列动态，其中请求到达和处理速率产生复杂的稳定性挑战，而纯优化方法无法解决这些问题。

贡献。本文做出了以下贡献：

C1) 据我们所知，我们提出了第一个基于李雅普诺夫辅助的深度强化学习框架，称为 Splitwise，用于动态 LLM 分区，它在保证系统稳定性的同时，联合优化边缘-云协作推理中的延迟和能耗（见第 3 节）。

C2) 我们提供了一种容错方法，使用分区边界检查点从通信故障中恢复，并以指数退避方式恢复执行。

C3) 我们引入了一种分层分区方案，该方案在注意力头和前馈子模块的层面上运行，暴露的分区点比传统的按层方法多出数百倍（参见第2.1节）。这种精细的粒度使得Splitwise能够在带宽和工作负载波动的情况下有效地适应。

C4) 我们通过在真实边缘设备（NVIDIA Jetson、手机）和各种大语言模型（1B-13B参数）上的广泛评估表明，与最先进的基线相比，Splitwise 将延迟降低了1.4×–2.8×，同时节能可达41%。此外，与仅云部署相比（参见第5节），Splitwise 可将运营推理成本降低多达6×。

论文结构。本文的其余部分组织如下。第二部分介绍我们的系统模型和问题表述。第三部分详述了基于Lyapunov的深度强化学习框架。第四部分描述了实现优化。第五部分评估了Splitwise。第六部分回顾相关工作。第七部分总结了本文。

2 系统模型与问题表述

我们首先介绍用于边缘-云协作大模型推理的系统模型，随后给出问题的公式化。表 1 总结了本文使用的关键符号。

2.1 边缘-云架构

系统组件。我们考虑一个边缘-云协作系统，它由一组边缘设备 $\mathcal{E} = \{E_1, E_2, \ldots, E_{N_e}$ 组成
每个边缘设备具有有限的计算资源，还有一组容量充足的云服务器 $C = \{C_1, C_2, \ldots, C_{N_c}\}$。每个边缘设备 $E_i \in \mathcal{E}$ 的特点是其计算能力 CC_e^i (FLOPs)、内存容量 M_e^i (GB) 以及功率预算 P_e^i (W)。云服务器提供显著更高的总体资源，即 $CC_c^j \gg CC_e^i$ 对大多数 j, i，但只能通过具有时变带宽 $B_i(t)$ 和延迟 $l_{n,i}(t)$ 的网络连接访问，这些延迟与每个边缘–云连接相关 [32]。

大型语言模型架构。我们考虑由基于 Transformer 的大型语言模型组成的 L 个顺序层。每一层 $\ell \in \{1, ..., L\}$ 处理输入张量 $X^{(\ell)} \in \mathbb{R}^{n \times d_{model}}$，其中 n 表示序列长度，d_{model} 为模型维度。层计算如下：

$$X^{(\ell+1)} = \text{FFN}^{(\ell)}(\text{MHA}^{(\ell)}(X^{(\ell)}) + X^{(\ell)}) + \text{MHA}^{(\ell)}(X^{(\ell)}) + X^{(\ell)} \tag{1}$$

多头注意力（MHA）模块分解成 H 个并行注意力头：

$$\text{MHA}^{(\ell)}(X) = \text{Concat}(\text{head}_1^{(\ell)}, ..., \text{head}_H^{(\ell)}) W_O^{(\ell)} \tag{2}$$

表 1. 本文中使用的主要符号总结

Notation	Description
System Architecture	
$\mathcal{E}, C$	Edge device and cloud server
CC_e, CC_c, M_e, P_e	Compute (FLOPS), memory (GB), and power (W) of edge/cloud
$B(t), l_n(t)$	Network bandwidth and latency at time t
Model Architecture	
L, H	Number of transformer layers and attention heads per layer
d_{model}	Model hidden dimension
d_h	Attention head dimension (d_{model}/H)
d_{ff}	Feed-forward network dimension
θ_i	Parameters of layer i
$X^{(\ell)}$	Input tensor to layer ℓ
Partitioning	
π	Complete partitioning strategy across all layers
$\pi^{(\ell)}$	Partition decision for layer ℓ
$\pi_h^{(\ell)}$	Placement of head h in layer ℓ (0: edge, 1: cloud)
π_{ℓ}^{FFN}	FFN partition mode (0: edge, 1: cloud, 2: split)
$\mathcal{E}_h^{(\ell)}, C_h^{(\ell)}$	Set of heads assigned to edge and cloud in layer ℓ
$\mathcal{B}(\pi)$	Set of partition boundaries
Performance Metrics	
$T(\pi, t)$	End-to-end inference latency under partition π
T_{comp}^e, T_{comp}^c	Computation time on edge and cloud
$T_{comm(\pi,t)}, E(\pi)$	Communication time and Energy consumption for partition π
$\Delta_{acc(\pi)}$	Accuracy degradation from partitioning
$Q_{b(\cdot)}$	Quantization function at boundary b
Queue Dynamics	
$Q(t)$	Queue backlog at time t
$\lambda(t)$,$\mu(\pi, t)$	Request arrival rate, Service rate under partition π
$A(t)$	New arrivals in time slot t
$L(Q(t))$, $\Delta(Q(t))$	Lyapunov function, Conditional Lyapunov drift
Reinforcement Learning	
s_t, a_t	System state and action (partition decision) at time t
$r(s_t, a_t)$	Reward function
π_θ	Policy network with parameters θ
$V_\phi^{perf}, V_\psi^{stab}$	Performance and Stability critic network
V	Lyapunov control parameter
$g(s_t, a_t)$	Immediate cost function
γ	Discount factor
τ_{temp}	Temperature for Gumbel-softmax

每个头 $h \in \{1, ..., H\}$ 独立计算：

$$\text{head}_h^{(\ell)} = \text{Attention}(XW_{Q,h}^{(\ell)}, XW_{K,h}^{(\ell)}, XW_{V,h}^{(\ell)}) \tag{3}$$

使用投影矩阵 $W_{Q,h}^{(\ell)}, W_{K,h}^{(\ell)}, W_{V,h}^{(\ell)} \in \mathbb{R}^{d_{model} \times d_h}$，其中 $d_h = d_{model}/H$。前馈网络（FFN）由两个线性变换及激活函数组成：

$$\text{FFN}^{(\ell)}(X) = \max(0, XW_1^{(\ell)} + b_1^{(\ell)})W_2^{(\ell)} + b_2^{(\ell)} \tag{4}$$

$W_1^{(\ell)} \in \mathbb{R}^{d_{model} \times d_{ff}}$和$W_2^{(\ell)} \in \mathbb{R}^{d_{ff} \times d_{model}}$之间，具有中间维度$d_{ff}$ =，通常为$4d_{model}$。

分区粒度。我们引入了一种分层分区方案，从而实现计算的灵活分配。设 $\pi = \{\pi^{(1)}, ..., \pi^{(L)}\}$ 表示完整的分区策略，其中每一层的分区 $\pi^{(\ell)}$ 定义为：

$$\pi^{(\ell)} = (\pi_{MHA}^{(\ell)}, \pi_{FFN}^{(\ell)}) \tag{5}$$

MHA 分区 $\pi_{MHA}^{(\ell)} = [\pi_{h_1}^{(\ell)}, ..., \pi_{h_H}^{(\ell)}] \in \{0, 1\}^H$ 指定了每个注意力头的放置位置，其中 $\pi_{h_i}^{(\ell)} = 0$ 表示边缘执行，$\pi_{h_i}^{(\ell)} = 1$ 表示云执行。FFN 分区 $\pi_{FFN}^{(\ell)} \in \{0, 1, 2\}$ 支持三种模式：

- $\pi_{FFN}^{(\ell)} = 0$: Entire FFN executes on edge

- $\pi_{FFN}^{(\ell)} = 1$: Entire FFN executes on cloud
- $\pi_{FFN}^{(\ell)} = 2$: Split execution with $W_1^{(\ell)}$ on edge and $W_2^{(\ell)}$ on cloud

该公式使每层可以有 $2^H \times 3$ 种可能的配置，从而产生总动作空间为 $(2^H \times 3)^L$ 个分区。对于具有 24 层且有 16 个头的模型，这大约产生 10^{31} 种可能的配置，因此需要智能探索策略 [13]。

数据流形式化。给定分区 $\pi^{(\ell)}$，通过层 ℓ 的数据流涉及潜在的边缘-云转换。设 $\mathcal{E}_h^{(\ell)} = \{h : \pi_h^{(\ell)} = 0\}$ 和 $C_h^{(\ell)} = \{h : \pi_h^{(\ell)} = 1\}$ 表示边缘和云端的头部分配 [32]。计算过程如下：

$$Y_{edge}^{(\ell)} = \sum_{h \in \mathcal{E}_h^{(\ell)}} \text{head}_h^{(\ell)}(X^{(\ell)}) W_{O,h}^{(\ell)} \tag{6}$$

$$Y_{cloud}^{(\ell)} = \sum_{h \in C_h^{(\ell)}} \text{head}_h^{(\ell)}(X^{(\ell)}) W_{O,h}^{(\ell)} \tag{7}$$

如果 $|\mathcal{E}_h^{(\ell)}| > 0$ 且 $|C_h^{(\ell)}| > 0$，则聚合需要通信，将大小为 $O(n \times d_{model})$ 的部分结果传输到网络上 [32]。

2.2 性能指标

延迟模型。分区 π 在时间 t 的端到端推理延迟 $T(\pi, t)$ 包含三个组成部分：

$$T(\pi, t) = T_{comp}^{e}(\pi) + T_{comm}(\pi, t) + T_{comp}^{c}(\pi) \tag{8}$$

其中 $T_{comp}^{e}(\pi) = \sum_{i=1}^{L} \sum_{j \in \mathcal{H}_i^e} \frac{F_{ij}}{C_e}$ 表示分配给具有 F_{ij} FLOPs 的边缘的头 $\mathcal{H}_i^e$ 的边缘计算时间，$T_{comm}(\pi, t) = \sum_{k=1}^{K(\pi)} \frac{D_k(\pi)}{B(t)} + l_n(t)$ 表示 $K(\pi)$ 次边缘-云转换的通信开销，数据量为 $D_k(\pi)$，而 $T_{comp}^{c}(\pi)$ 表示云计算时间。

能耗。边缘设备的能耗结合了计算和通信成本 [30]：

$$E(\pi) = \sum_{i=1}^{L} \sum_{j \in \mathcal{H}_i^e} P_e^{comp} \cdot t_{ij} + \sum_{k=1}^{K(\pi)} P_e^{comm} \cdot \frac{D_k(\pi)}{B(t)} \tag{9}$$

其中 P_e^{comp} 和 P_e^{comm} 分别表示用于计算和通信的功耗，t_{ij} 是第 i 层中组件 j 的执行时间 [30]。

精度保持。分区在边界引入量化以减少通信。令 Q_b 表示边界 b 的量化函数。精度下降建模为：

$$\Delta_{acc}(\pi) = \sum_{b \in \mathcal{B}(\pi)} \alpha_b \cdot \|x_b - Q_b(x_b)\|_2^2 \tag{10}$$

其中 $\mathcal{B}(\pi)$ 表示分区边界，x_b 是边界 b 处的激活张量，α_b 根据梯度流分析权重每个边界的重要性。

2.3 队列动态与稳定性

请求队列模型。推理请求根据速率 $\lambda(t)$ 的随机过程到达。我们对队列积压 $Q(t)$ 建模，表示时间 t 未处理请求的数量。队列演变如下：

$$Q(t+1) = \max[Q(t) - \mu(\pi, t), 0] + A(t) \tag{11}$$

其中 $\mu(\pi, t) = 1/T(\pi, t)$ 是分区 π 下的服务速率，$A(t)$ 表示时隙 t 的新到达 [1]。

李雅普诺夫函数。为了确保队列的稳定性，我们定义李雅普诺夫函数：

$$L(Q(t)) = \frac{1}{2} Q(t)^2 \tag{12}$$

条件李雅普诺夫漂移 $\Delta(Q(t))$ 衡量队列积压的预期变化：

$$\Delta(Q(t)) = \mathbb{E}[L(Q(t+1)) - L(Q(t)) | Q(t)] \tag{13}$$

2.4 问题表述

目标函数。我们将动态分区问题表述为在保持队列稳定的同时，最小化延迟、能耗和精度损失的加权组合：

$$\min_{\pi(t)} \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t \left(w_T T(\pi(t), t) + w_E E(\pi(t)) + w_A \Delta_{acc}(\pi(t))\right)\right] \tag{14}$$

受以下条件约束：

$$\limsup_{t \to \infty} \frac{1}{t} \sum_{\tau=0}^{t-1} \mathbb{E}[Q(\tau)] < \infty \quad \text{(queue stability)} \tag{15}$$

$$\sum_{j \in \mathcal{H}_i^e} m_{ij} \le M_e, \quad \forall i \quad \text{(memory constraint)} \tag{16}$$

$$E(\pi(t)) \le P_e \cdot T(\pi(t), t) \quad \text{(power constraint)} \tag{17}$$

其中 $\gamma \in (0, 1)$ 是折扣因子，w_T、w_E、w_A 是重要性权重，m_{ij} 表示第 i 层中组件 j 的内存需求。

受约束的MDP表述。我们将此优化问题表示为一个带有以下内容的受约束马尔可夫决策过程（MDP）：

- 状态空间 $\mathcal{S}$: $s_t = (Q(t), B(t), \lambda(t), \mathcal{R}(t))$ 捕获队列积压、网络带宽、到达率和资源可用性（详情见第3.1节）
- 动作空间 $\mathcal{A}$: $a_t = \pi(t)$ 表示分区决策（详情见第 3.1 节）
- 转变动态：$P(s_{t+1}|s_t, a_t)$ 由队列演化和随机网络/工作负载变化控制
- 奖励函数：结合第3.2节中详细说明的即时成本和李雅普诺夫漂移

挑战在于求解具有连续状态空间、指数级大的动作空间和稳定性要求的受约束MDP，我们通过我们的Lyapunov辅助强化学习框架来解决这一问题。

3 Splitwise 设计概述

Splitwise 框架将深度强化学习与李雅普诺夫优化理论相结合，以实现最优性能和保证的稳定性。该框架由三个关键组件组成：(i) 学习分区决策的策略网络 π_θ，(ii) 评估长期稳定性的李雅普诺夫评论器，以及 (iii) 平衡即时性能与队列稳定性的漂移加罚奖励函数。与可能收敛到不稳定策略的标准强化学习方法不同，Splitwise 明确将稳定性约束纳入学习过程（见图 2）。

3.1 状态与动作表示

状态编码。我们将系统状态 s_t 编码为一个全面的特征向量，涵盖瞬时条件和时间动态：

$$s_t = [\underbrace{Q(t), \bar{Q}_\tau}_{\text{queue}}, \underbrace{B(t), \bar{B}_\tau, \sigma_B}_{\text{network}}, \underbrace{\lambda(t), \bar{\lambda}_\tau}_{\text{workload}}, \underbrace{C_e^{avail}, M_e^{avail}}_{\text{resources}}, \underbrace{h_t}_{\text{history}}] \quad (18)$$

其中$\bar{Q}_\tau = \frac{1}{\tau}\sum_{i=t-\tau}^{t} Q(i)$ 表示窗口 τ 上的移动平均队列长度，σ_B 捕捉网络带宽的方差，$h_t \in \mathbb{R}^{d_h}$ 是一个从 LSTM 学习得到的历史嵌入，用于编码过去的分区决策及其结果 [6]。

层次化动作分解。为了处理指数级大的动作空间 $(2^H \times 3)^L$，我们对动作进行层次化分解。我们不是从所有可能的分区中选择，而是将动作结构化如下：

$$a_t = \pi_\theta(s_t) = [\alpha^{(1)}, ..., \alpha^{(L)}] \quad (19)$$

每一层的操作 $\alpha^{(\ell)}$ 都由以下方式生成：

$$\alpha^{(\ell)} = \text{softmax}(f_\theta^{(\ell)}(s_t, e^{(\ell-1)})) \quad (20)$$

这里，$f_\theta^{(\ell)}$ 是一个特定于层的子网络，$e^{(\ell-1)}$ 编码了来自前几层的决策以捕捉层间依赖。每个 $\alpha^{(\ell)} \in [0,1]^{H+1}$ 表示将每个注意力头和 FFN 放置在云端的连续概率，这些概率在执行时使用 Gumbel-softmax 进行离散化：

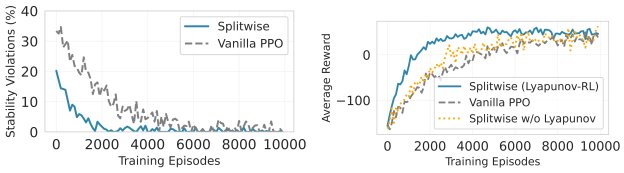
$$\pi_h^{(\ell)} = \text{Gumbel-Softmax}(\alpha_h^{(\ell)}, \tau_{temp}) \quad (21)$$

其中 τ_{temp} 是在训练过程中退火的温度参数，用于从探索过渡到利用。

3.2 李雅普诺夫引导的奖励设计

漂移加惩罚公式。传统的强化学习优化预期累积奖励，但不能保证稳定性。我们引入李雅普诺夫漂移以在优化性能的同时确保队列稳定性。奖励函数将即时成本与漂移惩罚结合起来：

$$r(s_t, a_t) = -[V \cdot \Delta(Q(t)) + g(s_t, a_t)] \quad (22)$$



(a) Training Stability (b) Learning Efficiency

图2. 模型训练动态的比较，显示了 (a) 在不同设置下的训练稳定性和 (b) 学习效率。

其中 $V > 0$ 是一个控制参数，$g(s_t, a_t)$ 表示即时成本：

$$g(s_t, a_t) = w_T T(\pi_t, t) + w_E E(\pi_t) + w_A \Delta_{acc}(\pi_t) \quad (23)$$

李雅普诺夫漂移计算。在动作 a_t 下的一步李雅普诺夫漂移为：

$$\begin{aligned}\Delta(Q(t)) &= \mathbb{E}[L(Q(t+1)) - L(Q(t)) \mid Q(t), a_t] \\ &\le B + Q(t)\,\mathbb{E}[A(t) - \mu(\pi_t, t)]\end{aligned} \quad (24)$$

其中 B 是一个有限常数，用于限制到达和服务的二阶矩。这个上界提供了一个可处理的优化目标，同时保持保证。

自适应权重调整。控制参数 V 在性能优化与稳定性之间进行平衡。我们根据队列积压自适应地调整 V：

$$V(t) = V_{min} + (V_{max} - V_{min}) \cdot \exp\left(-\frac{Q(t)}{Q_{ref}}\right) \quad (25)$$

其中 Q_{ref} 是一个引用队列长度。这确保在队列稳定时进行积极的性能优化，同时在积压增加时优先保证稳定性。

3.3 策略学习算法

演员-评论家架构。我们采用带有双评论家的近端策略优化（PPO）算法 [26]，用于分别评估性能和稳定性：

- *Performance critic* $V_\phi^{perf}(s)$: Estimates expected cumulative performance cost
- 稳定性评估器 $V_\psi^{stab}(s)$：估计预期队列积压演变

组合价值函数为：

$$V(s) = V_\phi^{perf}(s) + V \cdot V_\psi^{stab}(s) \quad (26)$$

带有稳定性约束的策略梯度。策略梯度同时包含性能和稳定性目标：

$$\nabla_\theta J = \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_t \nabla_\theta \log \pi_\theta(a_t|s_t) A_t^{total}\right] \quad (27)$$

总优势函数结合了两个评论者：

$$A_t^{total} = A_t^{perf} + V \cdot A_t^{stab} \quad (28)$$

使用广义优势估计（GAE）计算优势以降低方差。

算法概述。我们的训练过程将李雅普诺夫优化与深度强化学习结合，以联合优化性能并保证稳定性。算法1展示了核心训练循环，并详细解释了每个组件。第2-4行（轨迹收集）：

Algorithm 1: Lyapunov-Assisted DRL Training

Input: Policy network π_θ, critics $V_\phi^{perf}, V_\psi^{stab}$
Output: Optimized policy π_θ^*

1 **Initialize:** π_θ, critics $V_\phi^{perf}, V_\psi^{stab}$, $\mathcal{D} \leftarrow \emptyset$, $Q(0) = 0$, $\tau_{temp} = 1.0$, $V = V_{init}$;
2 **for** *episode* $k = 1, 2, ..., K$ **do**
3 $s_0, \tau_k \leftarrow$ GetInitialState(), $\emptyset$;
4 **for** *step* $t = 0, 1, ..., T_{max}$ **do**
5 $a_t \sim \pi_\theta(a|s_t)$; // Sample action from policy
6 $s_{t+1}, c_t \leftarrow$ ExecutePartition(a_t) ; // Execute partition
// Compute Lyapunov drift
7 $\mu_t \leftarrow 1/T(\pi_t, t)$; // Service rate
8 $\Delta(Q(t)) \leftarrow Q(t) \cdot (\lambda(t) - \mu_t)$;
// Calculate shaped reward
9 $r_t \leftarrow -[V(t) \cdot \Delta(Q(t)) + w_T c_t^T + w_E c_t^E + w_A c_t^A]$;
// Update queue dynamics
10 $Q(t+1) \leftarrow \max[Q(t) - \mu_t \cdot \delta_t, 0] + A(t)$;
11 $\tau_k \leftarrow \tau_k \cup \{(s_t, a_t, r_t, s_{t+1})\}$;
12 $s_t \leftarrow s_{t+1}$;
13 StoreTrajectory$(\mathcal{D}, \tau_k)$; // Add to replay buffer
14 **if** $|\mathcal{D}| \geq B_{min}$ ***and*** $k \mod U_{freq} = 0$ **then**
// Update critics
15 $\mathcal{B} \leftarrow$ SampleBatch$(\mathcal{D}, B_{size})$;
16 $\mathcal{L}^{perf} \leftarrow \frac{1}{|\mathcal{B}|} \sum_{(s,r)\in\mathcal{B}} (V_\phi^{perf}(s) - R^{perf})^2$;
17 $\mathcal{L}^{stab} \leftarrow \frac{1}{|\mathcal{B}|} \sum_{(s,Q)\in\mathcal{B}} (V_\psi^{stab}(s) - Q_{target})^2$;
18 $\phi \leftarrow \phi - \alpha_\phi \nabla_\phi \mathcal{L}^{perf}$;
19 $\psi \leftarrow \psi - \alpha_\psi \nabla_\psi \mathcal{L}^{stab}$;
// Update policy using PPO
20 **for** *PPO epoch* $e = 1, ..., E_{ppo}$ **do**
21 $A^{total} \leftarrow$ ComputeAdvantage$(\mathcal{B}, V_\phi^{perf}, V_\psi^{stab})$;
22 $\rho \leftarrow \pi_\theta(a|s)/\pi_{old}(a|s)$; // Importance ratio
23 $J_{PPO} \leftarrow \min(\rho A^{total}, \text{clip}(\rho, 1-\epsilon, 1+\epsilon)A^{total})$;
24 $\theta \leftarrow \theta + \alpha_\theta \nabla_\theta J_{PPO}$;
// Adapt control parameters
25 $V(t) \leftarrow$ UpdateControlParam(Q_{avg}, V) ;
26 $\tau_{temp} \leftarrow \beta \cdot \tau_{temp}$; // Anneal exploration
27 **return** π_θ

每一集模拟在不同网络条件下的一系列推理请求。策略 π_θ 根据当前系统状态生成分区决策，包括队列积压、网络带宽和资源可用性。第6行（分区执行）：所选分区 a_t 在边缘和云端执行，返回下一个状态以及延迟、能耗和精度的即时成本 $c_t = (c_t^T, c_t^E, c_t^A)$。第7-8行（Lyapunov漂移计算）：漂移表示队列积压的预期变化。正漂移表示队列增长（不稳定），而负漂移表示队列在消耗。这是确保系统稳定性的关键创新。第9行（奖励塑形）：奖励将即时性能成本与Lyapunov漂移惩罚结合。控制参数 $V(t)$ 根据当前队列状态动态平衡性能优化与稳定性。第10行（队列演化）：队列动态遵循Lindley递归，其中 δ_t 是时隙持续时间，$A(t)$ 表示遵循泊松过程的新到达。第15-19行（评论者更新）：两个独立的评论者学习预测性能成本和队列演变。性能评论者估计累积延迟/能量成本，而稳定性评论者预测未来的队列积压。两种方法都利用带有目标网络的时序差分学习以增强稳定性。第20-24行（PPO策略更新）：使用带有裁剪代理目标的近端策略优化更新策略，以防止破坏性更新。优势函数结合了两个评论者的预测，并按 $V(t)$ 加权。

第25行（控制参数自适应）：控制参数 V 根据平均队列长度进行调整，以在最大化性能的同时保持稳定性。

3.4 收敛性与稳定性分析

理论保证。在我们的框架下，我们建立了两个关键的理论结果：

定理 1（队列稳定性）：如果到达率 $\lambda < \mu_{max}$，其中 μ_{max} 是最大可实现服务率，则 Lyapunov 辅助的 RL 策略保证：

$$\limsup_{T\to\infty} \frac{1}{T} \sum_{t=0}^{T-1} \mathbb{E}[Q(t)] \leq \frac{B + V \cdot g^*}{\epsilon} \tag{29}$$

其中 g^* 是最优性能成本，$\epsilon = \mu_{max} - \lambda$ 是容量余量。

证明概要：通过将Lyapunov漂移加入奖励中，策略学会在队列变大时采取最小化漂移的动作，从而确保时间平均积压有界。

定理 2（性能界）：在我们的策略下，时间平均性能成本满足：

$$\limsup_{T\to\infty} \frac{1}{T} \sum_{t=0}^{T-1} \mathbb{E}[g(s_t, a_t)] \leq g^* + \frac{B}{V} \tag{30}$$

这表明性能随着 $V \to \infty$ 接近最优，但会以排队积压为代价。

3.5 在线适应

快速适应机制。在部署过程中，网络条件和工作负载可能与训练时不同。我们通过以下方式实现在线适应：

$$\theta_{adapted} = \theta_{base} + \alpha_{adapt} \nabla_\theta J_{online} \tag{31}$$

其中 J_{online} 是根据近期部署经验计算的，对稳定性给予更高权重，以防止在适应期间系统退化。

4 实施

4.1 系统架构

运行时组件。Splitwise 由四个主要的运行时组件组成，部署在边缘和云端，如图 3 所示：(1) 部署在边缘设备上的分区控制器，执行学习到的策略并协调执行，

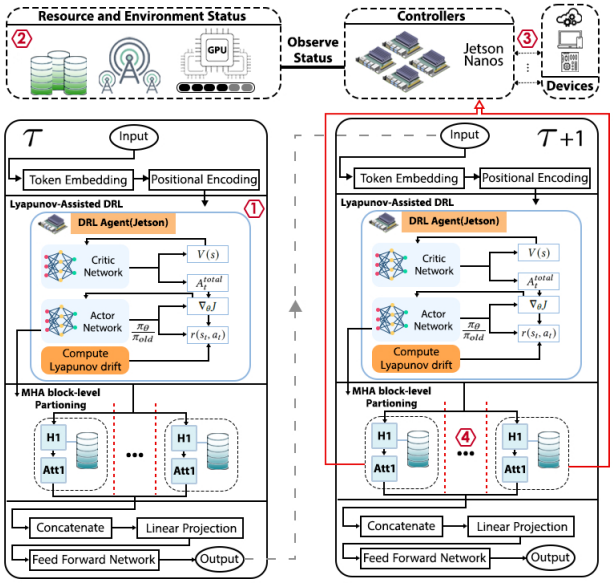


图 3. Splitwise 的系统架构。该框架由四个主要运行时组件组成（见第 4.1 节）：(1) 分区控制器，(2) 性能分析引擎，(3) 通信管理器，以及 (4) 执行运行时。

(2) 性能分析引擎，以最小开销收集性能指标，(3) 通信管理器，处理数据序列化和传输，并进行自适应压缩，(4) 在边缘和云端的执行运行时，管理模型分片的执行。控制器维护一个轻量级状态机，用于跟踪分区决策并同步边缘-云执行。

模型准备。为了实现细粒度的分区，我们在部署时修改模型架构，而无需重新训练。每个变压器层被分解为可独立执行的组件 [14]:

$$\text{Layer}_\ell = \{\text{Head}_1^{(\ell)}, ..., \text{Head}_H^{(\ell)}, \text{FFN}_1^{(\ell)}, \text{FFN}_2^{(\ell)}\} \quad (32)$$

我们实现了自定义的 CUDA 内核，使各个注意力头可以独立执行，同时保持与原始模型的数值等价性。与整体执行相比，这种分解带来的额外开销可忽略不计（<0.3 %）。

我们使用包含最近1000个测量的经验缓存 [22]，并每进行100次推理就更新一次预测器，以适应不断变化的系统条件。

异步流水线执行。我们实现了一个三阶段流水线以隐藏通信延迟 [3]：1）边缘计算分区 $\pi_{edge}^{(\ell)}$ 对于层 ℓ，然后 2）在将结果传输到云端的同时，边缘开始层 $\ell+$ 的计算，如果 $\pi_{edge}^{(\ell+1)} \neq \emptyset$，则 3）云处理接收到的数据，与边缘执行并行。对于边缘和云都有大量工作负载的平衡分区，该流水线可将有效延迟降低多达 35%。

动态模型加载。边缘设备无法将整个模型保存在内存中。我们实现了动态加载仅维护活动分区的方案：

$$M_{active} = \sum_{\ell=1}^{L} \sum_{h \in \mathcal{E}_h^{(\ell)}} m_h^{(\ell)} + \sum_{\ell:\pi_{FFN}^{(\ell)} \in \{0,2\}} m_{FFN}^{(\ell)} \quad (33)$$

模型分片从闪存存储中加载，并根据预测的未来分区进行预取，实现了单个组件<5毫秒的加载延迟。

分区检查点。我们在分区边界维护检查点，以便从通信故障中恢复：

$$\text{Checkpoint}_b = \{X_b, \pi_b, t_b\} \quad (34)$$

如果传输失败，执行将从上一个检查点恢复，并采用指数退避策略。

5 绩效评估

5.1 实验设置

硬件平台。我们在各种边缘设备上评估了 Splitwise，这些设备代表了不同的部署场景，从移动电话到物联网网关。表 2 总结了我们的硬件配置。这些边缘设备的计算能力跨度很大，从用于 AI 工作负载的强大 Jetson Orin NX，到资源受限的 Raspberry Pi 5，代表物联网场景。对于云基础设施，我们使用了一个大学集群，其配置类似于 AWS EC2 p4d.24xlarge 实例，配备 8 块 NVIDIA A100 GPU，提供 640GB GPU 内存和 2.4TB/s 内存带宽。此设置反映了现实的边缘-云部署场景，其中资源受限的设备与强大的云服务器协作。

网络状况。我们使用 Linux 流量控制（tc）模拟现实的网络环境，以调整边缘设备与云设备之间的带宽和延迟。表 3 展示了我们的网络配置，这些配置来源于在大学校园内收集的为期 3 天的真实 5G 和 WiFi 测量数据。这些配置涵盖了从极佳 WiFi 连接到恶化的移动网络条件的典型场景。

模型和数据集。表 4 详细列出了我们评估中使用的 LLM 架构。我们选择了跨越三个数量级的模型，以展示 Splitwise 的可扩展性。每种模型都带来了独特的挑战：GPT-2 1.5B [25] 可以完全放入边缘内存，但需要为延迟进行优化；LLaMA-7B[1] 需要仔细的内存管理和分区；而 LLaMA-13B[2] 如果没有我们的分区方法，则无法在边缘运行。不同的

[1]https://huggingface.co/dfurman/LLaMA-7B
[2]https://huggingface.co/dfurman/LLaMA-13B

表 2. 评估中使用的硬件平台

Type	Device	Memory	Compute	Power	Use Case
Edge	NVIDIA Jetson Orin NX	8GB	100 TOPS	25W	AI Gateway
	Samsung Galaxy S23	12GB	Snapdragon 8G2	8W	Mobile
	Raspberry Pi 5 + NPU	8GB	13 TOPS	12W	IoT Device
Cloud	AWS p4d.24xlarge	1.1TB	8×A100 GPUs	-	Server

表 3. 评估的网络条件。

Scenario	Bandwidth	Latency	Jitter	Loss Rate
Wifi	100 Mbps	10ms	2ms	0.01%
5G-(Good)	50 Mbps	20ms	5ms	0.1%
5G-(Average)	25 Mbps	40ms	10ms	0.5%
4G	10 Mbps	80ms	20ms	1%
Variable (Var) [15]	10-100 Mbps	10-100ms	2-20ms	0.01-1%

表 4. 模型配置和特征

Model	Parameters	Layers	Heads	Hidden	Memory
GPT-2 [25]	1.5B	24	16	1,600	6GB
LLaMA-7B	7B	32	32	4,096	28GB
LLaMA-13B	13B	40	40	5,120	52GB

表 5. Splitwise 超参数配置和训练设置

Category	Hyperparameter	Value
RL Training	Learning rate (actor)	3×10^{-4}
	Learning rate (critics)	1×10^{-3}
	Discount factor γ	0.99
	GAE parameter λ	0.95
	PPO clip range	0.2
	Entropy coefficient	0.01
Lyapunov Control	V range	[0.1, 10.0]
	V adaptation rate	0.1
	Reference queue Q_{ref}	10 requests
	Critical queue $Q_{critical}$	50 requests
	Drift bound B	100
Experience Replay	Replay buffer size	1,000 transitions
	Batch size	256
	Priority exponent α	0.6
	Update frequency	Every 100 steps
Training Process	Episodes	2,500
	Warm-up episodes	100
	Evaluation frequency	Every 50 episodes

人数（16-40）和层深（24-40）测试了我们框架处理多样化架构模式的能力。我们使用 LMSYS-Chat-1M 数据集 [33]，该数据集包含来自 Vicuna 演示和 ChatGPT 交互的一百万条真实世界对话。这是一种真实的推理工作负载，序列长度范围从 50 到 2048 个标记。

基线。为了确保公平比较，我们使用各基线报告的最佳配置进行实现。所有方法在评估过程中共享相同的硬件平台和网络条件。我们使用作者提供的官方实现（如有）或根据已发布的规范进行复现。所有实验使用相同的随机种子。表5详细列出了Splitwise的具体设置。

仅边缘：整个模型在边缘上运行，使用4位量化；仅云端：整个模型在云端运行，伴随网络传输；边缘分片：在第 $L/2$ 层进行固定分区 [32]；边缘流水线：使用静态优化的流水线并行 [10]；CE-LSLM：动态执行并支持提前退出 [34]。

指标。我们测量：(i) 端到端延迟（P50，P95，P99），(ii) 边缘设备的能耗，(iii) 数据集上的模型准确性，以及 (iv) 不同网络下的分区情况。

5.2 实验结果

消融研究与规模分析。表6中的消融研究和表7中的规模分析共同

表6. 消融研究：组件贡献

Configuration	Latency (ms)	Stability
Full `Splitwise`	87	Stable at 8.5 req/s
w/o Lyapunov drift	91	Unstable at 6 req/s
w/o Pipeline exec.	118	Stable at 8.5 req/s
Fixed V parameter	95	Stable at 7.2 req/s

展示了Splitwise框架的有效性、鲁棒性和可扩展性。表6通过测量端到端延迟和在不同配置下的系统稳定性来评估关键组件的贡献。完整的Splitwise系统在保持每秒8.5个请求的稳定性的同时，实现了87毫秒的延迟。当我们从奖励函数中移除Lyapunov漂移时，延迟增加了4.6%。更关键的是，在仅每秒6个请求的情况下系统不稳定，这突显了Lyapunov优化在确保动态工作负载下队列稳定性中的关键作用。禁用自适应V显著降低了性能，延迟增加了9%至95毫秒，因为系统失去了在分区边界最小化通信开销的能力。

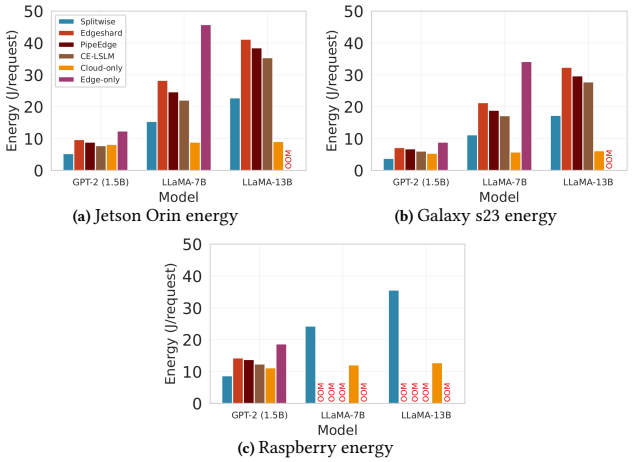
表7说明了Splitwise随着模型规模的增加是如何进行扩展的，从而在边缘设备上实现大型LLM的高效推理。随着模型从1.5B参数增长到13B参数，Splitwise通过将计算密集型组件卸载到云端，同时将内存和通信需求保持在边缘设备可行范围内，实现了加速提升（平均从1.4×到2.8×）。所需的边缘内存从2.1 GB增加到5.8 GB，仍低于现代边缘硬件的容量，而通信量随模型规模呈亚线性增长。如表7所示，准确率的下降很小（从GPT-2 1.5B到LLaMA-13B下降不到4%）。这种下降主要来自两个因素：(i) 仅在边缘-云边界使用的轻量级激活量化以降低传输成本，以及(ii) 在多头注意力输出的分布式重组合过程中引入的少量数值差异。

能源消耗。图4展示了Splitwise在三种具有不同计算能力的多样化边缘设备上的能效与最先进基线的全面分析：高性能的NVIDIA Jetson Orin NX（为AI工作负载设计）、移动设备Samsung Galaxy S23以及资源受限的Raspberry Pi 5。评估使用三种逐渐增大的大型语言模型（LLM）进行，以评估可扩展性。

结果一贯显示，Splitwise 的能耗低于所有竞争方法。在 Jetson Orin 上（图 4a），与 Edgeshard 和 PipeEdge 相比，Splitwise 的能耗最高可降低 41%，并且

表 7. 模型规模与性能的关系

Model Size	Speedup	Memory (GB)	Comm. (GB)	Accuracy
1.5B	1.4×	2.1	0.8	91.2%
7B	2.3×	4.2	2.1	90.5%
13B	2.8×	5.8	3.5	87.8%



(a) Jetson Orin energy (b) Galaxy s23 energy (c) Raspberry energy

图4. 在不同网络条件下，不同边缘设备和大语言模型的能耗对比。每次请求的能耗 (a) 在 NVIDIA Jetson Orin NX 上，(b) 在三星 Galaxy S23 智能手机上，(c) 在 Raspberry Pi 5 上。

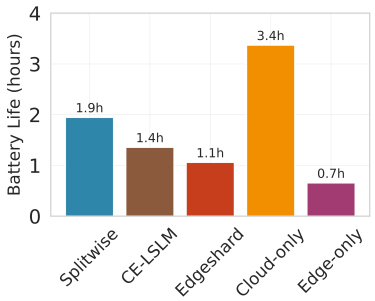
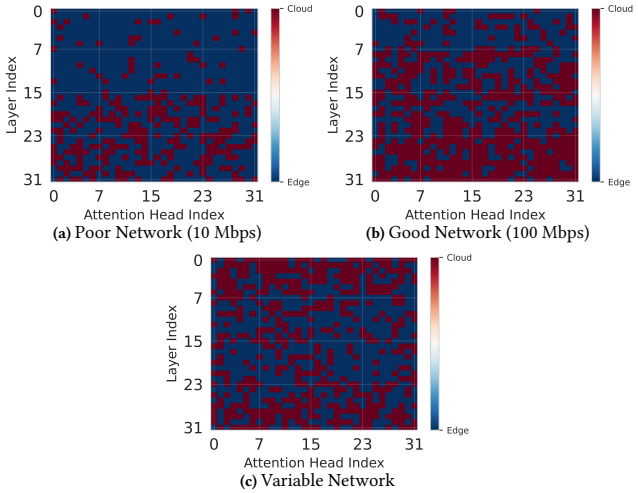


图5. 移动设备在每小时600次推理请求下的电池寿命。

与仅使用 Edge 的基线相比，LLaMA-7B 模型的能耗降低了超过 77%。这种显著的改进归功于其智能分区策略，该策略将计算密集型组件卸载到云端，从而减轻了边缘设备的负载。仅使用云的方法由于本地计算很少而表现出低能耗，但由于隐私问题和高延迟，对于敏感应用来说并不实用。仅使用 Edge 的基线，特别是对于像 LLaMA-7B 和 LLaMA-13B 这样较大的模型，会产生非常高的能耗，因为它必须在本地执行整个模型，而无法利用云端更强的计算资源。在 Galaxy S23（图 4b）上，趋势类似，Splitwise 的能耗低于所有其他方法。值得注意的是，对于较大的模型，节能效果更加明显，突显了协作推理在移动设备上实现高能效执行的优势。Raspberry Pi 5（图 4c）显示



(a) Poor Network (10 Mbps) (b) Good Network (100 Mbps) (c) Variable Network

图6. Splitwise在(a)网络状况差、(b)网络状况良好和(c)网络状况可变的情况下学习到的动态和细粒度分区策略。每个热图展示了跨层的Transformer注意力头的放置位置，其中蓝色表示边缘执行，红色表示云端执行。

因为动态加载和分区过程会引入额外的开销，系统必须经常在闪存和内存之间交换模型分片。

图5展示了不同推理策略对移动设备电池寿命的影响，这是用户体验和实际部署的关键因素。在每小时600次请求的连续工作负载下，仅云端的基线方案实现了最长的电池寿命，为3.4小时。这是因为它将所有计算任务卸载到云端，从而最小化了边缘设备上的本地CPU和GPU使用[27]。然而，这种方法会引入显著的延迟和成本。

网络。图6中的热力图展示了Splitwise在不同网络场景下学习到的细粒度分区策略，突显了其动态适应能力。在带宽为10 Mbps的较差网络条件下（图6a），策略表现出强烈的边缘执行倾向，这从大量蓝色像素可以看出。这种保守策略能够将通信开销降至最低，这在网络带宽有限时尤为关键。通过保持本地计算，Splitwise优先考虑低延迟，并减少与数据传输相关的能耗，有效缓解将任务卸载到云端可能带来的显著延迟。相反，在带宽为100 Mbps的良好网络条件下（图6b），分区策略变得更加平衡和灵活。红色与蓝色区域的复杂交错表明，代理利用高带宽链接进行计算卸载。

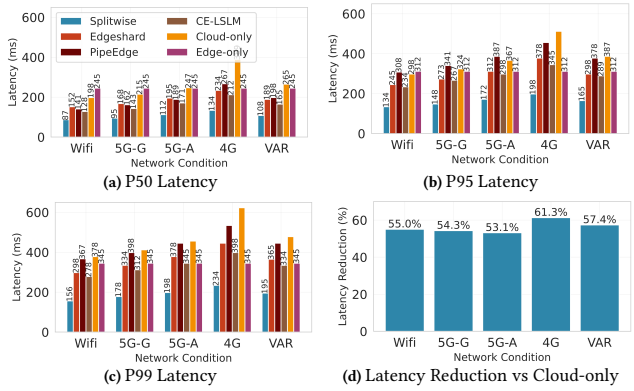


图7. 在不同网络条件下对Splitwise性能的评估。 (a) 和 (c) 比较了 Splitwise 与最先进基线在不同网络场景下的中位数（P50）和99百分位（P99）端到端推理延迟。 (b) 展示了 P95 延迟结果。 (d) 量化了 Splitwise 相较于仅云基线在 P95 延迟上实现的百分比减少。

最后，在可变网络场景（图6c）中，该场景模拟了现实世界中连接的波动，分区模式反映了一种偏向边缘执行的谨慎策略。这种行为是由Splitwise中的Lyapunov辅助驱动的，该辅助在奖励函数中引入了稳定性保证。智能体学习优先考虑队列的稳定性，即使在网络质量较差的时期，也能确保延迟有界。

延迟。图7显示了Splitwise在减少延迟的同时保持系统稳定性的有效性。在图7a和图7b中显示的P50（中位数）和P99（第99百分位）延迟指标中，Splitwise在所有网络场景下始终优于所有竞争基线。仅云方法由于通信开销显著，表现出最高的延迟，尤其在4G和VAR等网络状况较差的情况下，延迟可能超过500毫秒。相比之下，仅边缘执行虽然避免了网络延迟，但由于边缘设备的处理能力有限，计算延迟较高。静态分区方法如Edgeshard和PipeEdge的表现优于上述极端情况，但仍无法适应动态网络波动，导致性能不佳。Splitwise的优势最明显体现在即使在挑战性条件下也能保持低且稳定的延迟。这是通过其

6 相关工作

静态模型划分。早期关于深度神经网络（DNN）划分的工作 [14, 17] 在边缘和云之间开创了按层拆分的方法。Neurosurgeon [14] 离线分析每层的延迟和能耗成本，并选择单一的最优拆分点。DADS [9] 扩展了这一方法，同时考虑多个 DNN，但仍然局限于静态划分。这些方法从根本上假设网络条件稳定且工作负载均匀，而在实际部署中带宽会随日变化 [8]。此外，现有方法在层粒度进行划分，错过了在 Transformer 块内进行更细粒度优化的机会。

边缘-云协同推理。协同推理系统 [5, 7, 11, 12, 16, 19, 34] 将计算分布在边缘和云资源之间。SPINN [16] 使用早期退出逐步优化预测，但这种方法与自回归大型语言模型不兼容，因为每个标记依赖于完整的模型执行。CoDL [11] 动态调整卷积神经网络的分割点，但依赖于启发式策略，这些策略无法在不同的模型架构之间泛化。

Splitwise 功能。我们的工作在三个方面不同于以往的文献。首先，我们在子层粒度上实现了细粒度分区，暴露出数量级更多的分区点。其次，我们通过 Lyapunov 分析提供了队列稳定性的理论保证，同时优化多个目标。第三，我们通过分层分解和学习嵌入来处理动作空间的组合爆炸，使该方法对大型模型可行。

7 结论

Splitwise 展示了具有队列稳定性保障的细粒度划分能够实现高效的边缘–云 LLM 推理。它将 Transformer 层分解为注意力头和前馈块，并使用带有 Lyapunov 辅助奖励的分层策略将组件分配到边缘或云端。这种设计比按层划分的方法提供了更多的拆分选项，同时通过动作分解保持可处理性。它能够适应波动的网络连接，与静态和仅云端基线相比，可实现 1.4×–2.8× 的延迟降低，最高 41% 的能耗节约，以及 53–61% 的 P95 延迟降低。它在即刻性能与长期队列稳定性之间取得平衡。未来工作包括集成早期退出和压缩，以进一步减少通信并扩大适用性。

致谢

本文由斯洛伐克共和国复苏与韧性计划资助，资助范围为“转型与创新联合体”项目（项目代码：09I02-03-V01-00012）。

参考文献

[1] Sohee Bae, Seungyul Han, 和 Youngchul Sung. 2020. Lyapunov 优化的强化学习表述：在队列稳定性边缘计算系统中的应用。arXiv 预印本 arXiv:2012.07279 (2020)。 [2] Gordon Owusu Boateng 等人. 2025. 关于通信、网络和服务管理的大型语言模型的综述：应用见解、挑战和未来方向。IEEE 通信综述教程 (2025)。 [3] Branden Butler, Sixing Yu, Arya Mazaheri, 和 Ali Jannesari. 2024. Pipeinfer：使用异步流水线推测加速 LLM 推理。发表于 SC24：高性能计算、网络、存储与分析国际会议。IEEE, 1–19。 [4] Barbara M. Chapman, Thomas Fahringer, 和 Hans P. Zima. 1994. 分布式内存多处理器系统上数据分布的自动支持。收录于《并行计算的语言与编译器》, Utpal Banerjee, David Gelernter, Alex Nicolau, 和 David Padua (编辑)。Springer Berlin Heid

[17] En Li, Zeng Liekang, Zhou Zhi, 和 Chen Xu. 2019. 边缘人工智能：通过边缘计算按需加速深度神经网络推理。arXiv:1910.05316 [cs.NI] https://arxiv.org/abs/1910.05316 [18] Senyao Li 等. 2025. 边缘 SLM 与云端 LLM 之间的协作推理与学习：算法、执行与开放挑战的综述。arXiv 预印本 arXiv:2507.16731 (2025). [19] May Malka, Erez Farhan, Hai Morgenstern, 和 Nir Shlezinger. 2022. 通过边缘集群实现去中心化低延迟协作推理。arXiv:2206.03165 [cs.LG] https://arxiv.org/abs/2206.03165 [20] Avanika Narayan, Dan Biderman, Sabri Eyuboglu, Avner May, Scott Linderman, James Zou, 和 Christopher Re. 2025. Minions：设备端与云端语言模型之间的成本高效协作。arXiv 预印本 arXiv:2502.15964 (2025). [21] Hyeonho Noh, Byonghyo Shim, 和 Hyun Jong Yang. 2025. 在动态无线环境中利用大语言模型进行自适应资源分配优化。